ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 07

Datos y operadores, ahora con calma

- Las variables son cajas con etiqueta
- Los cinco tipos de datos básicos
- Textos: comillas simples, dobles y llaves
- Constantes: cajas grabadas en piedra
- Cuando PHP convierte tipos por su cuenta
- Comparar bien: == frente a ===
- null, el operador ?? y el ternario
- Operadores lógicos y paréntesis
- Redondear y mostrar números
- Los errores típicos de hoy



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En las clases 4 a 6 fuimos deprisa a propósito: quería que vierais PHP funcionando cuanto antes. Ahora toca lo contrario. **Vamos despacio**, volvemos a los datos y a los operadores, y nos paramos en los detalles que más errores provocan al empezar.

Os aviso: hoy no hay nada «nuevo y espectacular». Hay algo mejor. Vais a entender **por qué** PHP se comporta como se comporta, y eso os ahorrará muchas horas de buscar fallos.

•
👌 Cómo vamos a trabajar a partir de ahora



1
🤔 Las variables son cajas con etiqueta



2
👍 Los cinco tipos de datos básicos



3
📘 Textos: comillas simples, dobles y llaves



4
👌 Constantes: cajas grabadas en piedra



5
😮 Cuando PHP convierte tipos por su cuenta



6
⚠️ Comparar bien: == frente a ===



7
🔍 null, el operador ?? y el ternario



8
🚀 Operadores lógicos y paréntesis



9
👌 Redondear y mostrar números



10
🐞 Los errores típicos de hoy



11
🚀 Ejercicios



12
👋 Lo que tenéis que llevaros de hoy





CLASE 07

👌 Cómo vamos a trabajar a partir de ahora

Desde esta clase hasta la 12 trabajaremos en un proyecto nuevo, `php-paso-a-paso`, con una subcarpeta por clase:

1. En Laragon, pulsad **Raíz** y cread la carpeta `php-paso-a-paso`.
2. Dentro, cread la carpeta `clase07`.
3. Pulsad **Recargar** en Laragon para que cree el virtual host.
4. Abrid `php-paso-a-paso` en VS Code.
5. Cada archivo se abrirá en `http://php-paso-a-paso.test/clase07/nombre-del-archivo.php`.

El método es el de siempre: **leer, escribir el ejemplo vosotros y comprobarlo**. En cada clase tenéis dos carpetas:

Carpeta	Para qué sirve
`ejemplos/`	Los ejemplos de la explicación, para comparar si algo no os funciona
`soluciones/`	Una solución posible de cada ejercicio. **Miradla solo después de intentarlo**



💡 Una solución «posible»: si la vuestra es distinta y funciona, está bien. En programación casi nunca hay una única forma de hacer las cosas.



CLASE 07 · PUNTO 1

🤔 Las variables son cajas con etiqueta

Imaginad una mudanza. Cada caja lleva una **etiqueta** escrita con rotulador («libros», «cocina») y dentro hay **algo**. Una variable es exactamente eso:

Esquema

```
$nombre        $edad        $apodo

┌──────────┐   ┌──────┐     ┌────────┐
│ "Lucía"  │   │  25  │     │ (nada) │
└──────────┘   └──────┘     └────────┘

etiqueta +     etiqueta +   la caja existe,
contenido      contenido    pero está vacía
```

- La **etiqueta** es el nombre de la variable, siempre con `$` delante.
- El **contenido** es el valor que guardáis con `=`.
- Podéis **cambiar el contenido** cuando queráis: `$edad = 26;` tira el 25 y guarda el 26.

Pero ojo: la analogía tiene un límite. En una caja real podéis meter varias cosas; en una variable sencilla, **solo un valor**. Para guardar varios valores juntos usaremos arrays (clase 10).



CLASE 07 · PUNTO 2

👍 Los cinco tipos de datos básicos

Cada contenido tiene un **tipo**: no es lo mismo un número que un texto. PHP lo deduce solo a partir del valor [1].

01-tipos.php — php-paso-a-paso/clase07 — Visual Studio Code

01-tipos.php

```php
<?php
$edad = 25;            // int: número entero
$altura = 1.72;        // float: número con decimales (¡con punto!)
$nombre = "Lucía";     // string: texto
$esAlumna = true;      // bool: verdadero o falso
$apodo = null;         // null: la caja existe, pero está vacía

var_dump($edad);       // int(25)
var_dump($nombre);     // string(6) "Lucía"
echo gettype($edad);   // integer
```

📄 Ejemplo: `01-tipos.php`

Tipo	Qué guarda	Ejemplo	Lo que muestra `var_dump()`
`int`	Enteros, positivos o negativos	`25`, `-3`	`int(25)`
`float`	Números con decimales	`1.72`, `19.99`	`float(1.72)`
`string`	Texto entre comillas	`"Lucía"`	`string(6) "Lucía"`
`bool`	Solo `true` o `false`	`true`	`bool(true)`
`null`	Ausencia de valor	`null`	`NULL`

Dos curiosidades que os vais a encontrar en el ejemplo:

- `"Lucía"` tiene 5 letras, pero `var_dump()` dice `string(6)`. No es un error: cuenta **bytes**, y la «í» ocupa dos. Volveremos a esto con los textos en la clase 10.

Y una más que confunde mucho: si hacéis `echo` de una variable que vale `false`, **no aparece nada** en pantalla. `echo` convierte `false` en un texto vacío. Para ver lo que hay de verdad dentro de la caja, `var_dump()` [2]. - `gettype()` llama `double` a los `float`. Es un nombre antiguo; significa lo mismo.



CLASE 07 · PUNTO 3

📘 Textos: comillas simples, dobles y llaves

Buena parte del trabajo de una web es **montar textos**: «Hola, Laura», «Tienes 3 productos en el carrito»... Por eso conviene tener claro cómo se escriben [3].

Comillas	Qué hace PHP	`'Hola $alumna'` / `"Hola $alumna"`
Simples `' '`	Nada: el texto sale **tal cual**	`Hola $alumna`
Dobles `" "`	Busca variables dentro y las **sustituye**	`Hola Laura`

02-comillas.php — php-paso-a-paso/clase07 — Visual Studio Code

02-comillas.php

```php
<?php
$alumna = "Laura";
$modulo = "PHP 8";

echo 'Bienvenida $alumna al curso de $modulo';     // literal
echo "Bienvenida $alumna al curso de $modulo";     // sustituye
echo "Bienvenida {$alumna} al curso de {$modulo}"; // sustituye (con llaves)
echo "Bienvenida " . $alumna . " al aula.";        // concatenando con el punto
```

¿Para qué sirven las **llaves** `{$variable}`? Para que PHP sepa exactamente dónde empieza y dónde acaba la variable:

02-comillas.php — php-paso-a-paso/clase07 — Visual Studio Code

02-comillas.php

```php
<?php
$cantidad = 3;
$unidad = "kilo";
echo "Compra {$cantidad} {$unidad}s de naranjas";   // Compra 3 kilos de naranjas
```

Sin llaves, PHP buscaría una variable llamada `$unidads`... que no existe. 😅



🎯 **Mi recomendación:** comillas dobles con llaves cuando haya variables dentro; comillas simples cuando el texto sea fijo.

📄 Ejemplo: `02-comillas.php` (incluye cómo poner comillas dentro de un texto)



CLASE 07 · PUNTO 4

👌 Constantes: cajas grabadas en piedra

Si una variable es una caja cuyo contenido puede cambiar, una **constante** es una caja **grabada en piedra**: le dais un valor al principio y nadie puede cambiarlo después [4].

03-constantes.php — php-paso-a-paso/clase07 — Visual Studio Code

03-constantes.php

```php
<?php
const IVA = 0.21;                        // forma recomendada
define("NOMBRE_TIENDA", "La Tiendita");  // forma clásica

echo "Bienvenidos a " . NOMBRE_TIENDA;
echo "Precio con IVA: " . 100 * (1 + IVA) . " €";    // 121 €
```

Regla	Ejemplo
No llevan `$`	`IVA`, no `$IVA`
Se escriben en MAYÚSCULAS (por convenio)	`NOMBRE_TIENDA`
No se pueden cambiar	`IVA = 0.10;` da error

¿Cuándo usarlas? Para datos que **no deben cambiar nunca** mientras se ejecuta el programa: el IVA, el nombre de la web, el número máximo de intentos...

📄 Ejemplo: `03-constantes.php`



CLASE 07 · PUNTO 5

😮 Cuando PHP convierte tipos por su cuenta

PHP es un lenguaje **flexible**: si le dais un texto donde espera un número, intenta convertirlo. A veces es cómodo. Otras veces da sorpresas [5].

04-conversion.php — php-paso-a-paso/clase07 — Visual Studio Code

04-conversion.php

```php
<?php
$a = "5" + 3;          // el + suma números → int(8)
$b = "5" . 3;          // el . une textos   → string(2) "53"
```

Fijaos bien: **el operador decide**. El `+` dice «quiero números» y el `.` dice «quiero texto». Mismos datos, resultado distinto.

Convertir nosotros: el *casting*

Podemos forzar la conversión poniendo el tipo entre paréntesis:

04-conversion.php — php-paso-a-paso/clase07 — Visual Studio Code

04-conversion.php

```php
<?php
$precio = (float) "19.99";    // float(19.99)
$entero = (int) 7.9;          // int(7)  ← ¡corta, no redondea!
$texto  = (string) 42;        // string(2) "42"
```



⚠️ `(int) 7.9` da **7**, no 8. Si queréis redondear, usad `round()` (lo vemos en el punto 7).

¿Qué es «falso» para PHP?

Cuando PHP necesita un `bool` (por ejemplo, en un `if`), convierte el valor. Solo estos valores se consideran **falsos**:

Valor	¿Falso?
`false`	✅
`0` y `0.0`	✅
`""` (texto vacío)	✅
`"0"` (el texto cero)	✅ ← este pilla a todo el mundo
`null`	✅
`[]` (array vacío)	✅

Todo lo demás es verdadero: `"hola"`, `-1`, `" "` (un espacio) e incluso el texto `"false"`.

📄 Ejemplo: `04-conversion.php`



CLASE 07 · PUNTO 6

⚠️ Comparar bien: == frente a ===

Este es, probablemente, el punto más importante de hoy.

Operador	Qué compara	`5 == "5"` / `5 === "5"`
`==`	Solo el valor (convierte tipos si hace falta)	`true`
`===`	El valor **y** el tipo	`false`
`!=`	Distinto valor	
`!==`	Distinto valor **o** distinto tipo	

Una analogía: `==` es un portero de discoteca que solo mira la cara; `===` además os pide el DNI. 😄

¿Por qué importa tanto? Porque **los formularios y la URL siempre envían texto**. Mirad este caso real:

05-comparar.php — php-paso-a-paso/clase07 — Visual Studio Code

05-comparar.php

```php
<?php
$edadDelFormulario = "18";       // llega como texto

if ($edadDelFormulario === 18) { // "18" no es 18: nunca entra
    echo "Tiene 18 años";
}

// Lo correcto: convertir primero y comparar después
$edad = (int) $edadDelFormulario;
if ($edad === 18) {
    echo "Ahora sí: tiene 18 años";
}
```

📄 Ejemplo: `05-comparar.php`



🎯 **Mi recomendación:** usad siempre `===` y `!==`. Si los tipos no coinciden, convertid vosotros antes de comparar. Así el código hace lo que leéis, sin conversiones escondidas [6].



CLASE 07 · PUNTO 7

🔍 null, el operador ?? y el ternario

`null` significa «aquí no hay nada». No es cero ni un texto vacío: es **ausencia de valor** [7].

¿Y para qué lo queremos? Para casos tan normales como un dato que el usuario no ha rellenado. Aquí entra el **operador de fusión de null**, `??`, que significa: «usa esto y, si no existe o es `null`, usa lo otro».

06-null-ternario.php — php-paso-a-paso/clase07 — Visual Studio Code

06-null-ternario.php

```php
<?php
$apodo = null;
echo $apodo ?? "Sin apodo";     // Sin apodo

// Su uso estrella: leer datos de la URL sin errores
// Probad: 06-null-ternario.php?nombre=Ana
$nombre = $_GET["nombre"] ?? "invitado";
echo "Hola, " . htmlspecialchars($nombre);
```

Sin `??` tendríais que escribir un `if` con `isset()` cada vez. Comparad:

06-null-ternario.php — php-paso-a-paso/clase07 — Visual Studio Code

06-null-ternario.php

```php
<?php
if (isset($_GET["nombre"])) {
    $nombre = $_GET["nombre"];
} else {
    $nombre = "invitado";
}
```

Cinco líneas contra una. Así de simple.

El operador ternario: un if/else de una línea

Otro atajo que veréis muchísimo [6]:

06-null-ternario.php — php-paso-a-paso/clase07 — Visual Studio Code

06-null-ternario.php

```php
<?php
// (condición) ? valor si es verdad : valor si es mentira
$mensaje = ($edad >= 18) ? "Pase adelante" : "Acceso denegado";

// Perfecto para mostrar un bool de forma legible
echo "Matriculada: " . ($matriculada ? "sí" : "no");
```

Usadlo solo para elecciones **sencillas** entre dos valores. Si necesitáis más, volved al `if` de toda la vida.

📄 Ejemplo: `06-null-ternario.php`

✅ Dos pruebas para comprobarlo

Haced esto	Debéis obtener
Abrid `06-null-ternario.php` sin nada más	Hola, invitado
Añadid `?nombre=Ana` al final de la dirección	Hola, Ana



Ya los visteis en la clase 5. Hoy añadimos lo que os faltaba: **el orden en que se evalúan**.

07-logicos-precedencia.php — php-paso-a-paso/clase07 — Visual Studio Code

07-logicos-precedencia.php

```php
<?php
$edad = 20;
$tieneEntrada = true;
$esVip = false;

if ($edad >= 18 && $tieneEntrada) { echo "Puede entrar"; }  // Y
if ($esVip || $tieneEntrada)      { echo "Tiene acceso"; }  // O
if (!$esVip)                      { echo "No es VIP"; }     // NO
```

Igual que en matemáticas, unos operadores van antes que otros [8]:

07-logicos-precedencia.php — php-paso-a-paso/clase07 — Visual Studio Code

07-logicos-precedencia.php

```php
<?php
echo 2 + 3 * 4;       // 14: primero la multiplicación
echo (2 + 3) * 4;     // 20: los paréntesis mandan
```

Con `&&` y `||` pasa lo mismo: `&&` se evalúa antes que `||`. ¿Os vais a acordar de eso dentro de un mes? Yo prefiero no depender de mi memoria:

07-logicos-precedencia.php — php-paso-a-paso/clase07 — Visual Studio Code

07-logicos-precedencia.php

```php
<?php
// Descuento para menores de 26 o mayores de 65, siempre que tengan entrada
$descuento = ($edad < 26 || $edad >= 65) && $tieneEntrada;
```



🎯 **Regla práctica:** cuando mezcléis `&&` y `||`, **poned paréntesis siempre**. Cuestan dos caracteres y evitan muchos disgustos.



⚠️ PHP también acepta `and` y `or` escritos con letras, pero **no los uséis**: se evalúan *después* del `=` y producen resultados absurdos. Lo veréis en el punto 10.

📄 Ejemplo: `07-logicos-precedencia.php`



CLASE 07 · PUNTO 9

👌 Redondear y mostrar números

Calculad el precio de algo de 59,90 € con IVA y PHP os dirá `72.479`. Correcto… pero nadie paga 72,479 €. Necesitamos **redondear** y **mostrar** el número como se hace en España.

08-redondeo-formato.php — php-paso-a-paso/clase07 — Visual Studio Code

08-redondeo-formato.php

```php
<?php
$total = 59.90 * 1.21;          // 72.479

echo round($total, 2);          // 72.48
echo floor(7.9);                // 7  → siempre hacia abajo
echo ceil(7.1);                 // 8  → siempre hacia arriba

// number_format(número, decimales, separador decimal, separador de miles)
echo number_format($total, 2, ",", ".") . " €";   // 72,48 €
```

Función	Qué hace	Ejemplo	Resultado
`round()`	Redondea a los decimales indicados [9]	`round(72.479, 2)`	`72.48`
`floor()`	Redondea hacia abajo	`floor(7.9)`	`7`
`ceil()`	Redondea hacia arriba	`ceil(7.1)`	`8`
`number_format()`	Devuelve un **texto** con formato [10]	`number_format(1234.5, 2, ",", ".")`	`"1.234,50"`
`intdiv()`	División entera	`intdiv(17, 5)`	`3`
`%`	Resto de la división	`17 % 5`	`2`



⚠️ `number_format()` devuelve **texto**. Usadlo solo para mostrar el resultado, al final. Si seguís calculando con él, os llevaréis sorpresas.

📄 Ejemplo: `08-redondeo-formato.php`



CLASE 07 · PUNTO 10

🐞 Los errores típicos de hoy

Error	Qué pasa	Cómo se escribe bien
Olvidar el `$`	`edad = 25;` → *syntax error*	`$edad = 25;`
Guiones en el nombre	`$mi-variable` se lee como `$mi` **menos** `variable`	`$miVariable` (*camelCase*)
`echo` de un `false`	No sale nada en pantalla	`var_dump($activo);`
Usar `or` en vez de `||`	`$x = false or true;` guarda `false`	`$x = false || true;`
Comparar con `=`	`0 = "0"` es `true` aunque los tipos no coincidan	`===`

📄 Ejemplo: `09-errores-tipicos.php` . Tiene comentadas las versiones con error: quitad las `//` una a una y leed el mensaje que os da PHP. Aprender a leer errores es la mitad del oficio.



CLASE 07 · PUNTO 11

🚀 Ejercicios

Cread los archivos en `php-paso-a-paso/clase07/`. Primero intentadlo; después comparad con la carpeta `soluciones/`.

- ☐ **01 · Ficha.** Guardad nombre, edad, nota y si está matriculado. Mostrad cada dato junto a su tipo con `gettype()`. → `ej01-ficha.php`
- ☐ **02 · Predicciones. Sin ejecutar**, escribid en un papel qué mostrará `var_dump()` con: `"10" + 5`, `"10" . 5`, `(int) "3.99"`, `(bool) "0"`, `10 == "10"` y `10 === "10"`. Después comprobadlo. → `ej02-predicciones.php`
- ☐ **03 · Ticket de compra.** 3 auriculares a 24,95 €. Mostrad la base, el IVA (21 %) y el total con formato español. → `ej03-ticket.php`
- ☐ **04 · Saludo por la URL.** Leed `nombre` y `edad` de la URL con `??`. Si no hay edad, decidlo; si la hay, indicad si es mayor de edad o cuántos años le faltan. → `ej04-saludo-url.php`
- ☐ **05 · Segundos a horas.** Convertid 4000 segundos en horas, minutos y segundos con `intdiv()` y `%`. Debe salir 1 h, 6 min y 40 s. → `ej05-tiempo.php`



🏆 **Reto:** en el ejercicio 04, probad con `?edad=abc`. ¿Qué ocurre y por qué? Pista: mirad qué devuelve `(int) "abc"`.



CLASE 07 · PUNTO 12

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Tipos	`int`, `float`, `string`, `bool` y `null`. `var_dump()` os dice cuál tenéis
Comillas	Dobles con llaves `"{$var}"` si hay variables; simples si el texto es fijo
Constantes	Sin `$`, en mayúsculas y no cambian: `const IVA = 0.21;`
Conversión	El operador decide: `+` quiere números y `.` quiere texto
Casting	`(int)`, `(float)` y `(string)` convierten a mano. `(int)` corta, no redondea
Comparar	Usad `===` y `!==`. Los formularios siempre envían texto
`??`	Valor por defecto si algo no existe o es `null`
Ternario	`(condición) ? sí : no` para elegir entre dos valores
Paréntesis	Si mezcláis `&&` y `||`, ponedlos siempre
Números	`round()` para calcular, `number_format()` solo para mostrar

📚 Fuentes y documentación oficial

1. Manual de PHP: tipos
2. Manual de PHP: var_dump
3. Manual de PHP: strings (cadenas de caracteres)
4. Manual de PHP: constantes
5. Manual de PHP: manipulación de tipos
6. Manual de PHP: operadores de comparación
7. Manual de PHP: null
8. Manual de PHP: precedencia de operadores
9. Manual de PHP: round
10. Manual de PHP: number_format